Exploratory Data Analysis (EDA) - Study Notes

Original study notes prepared for the DataMind RAG Assistant project.

What is exploratory data analysis?

Exploratory data analysis (EDA) is the process of investigating a dataset — before formal modeling — to understand its structure, spot patterns, detect anomalies, and check underlying assumptions, using a combination of summary statistics and visualizations.

EDA typically comes right after data collection and cleaning, and its findings shape everything downstream: which features are worth engineering, which model family might fit the data, and which data-quality issues need to be fixed first.

Why data cleaning matters

Real-world data is rarely ready to analyze as-is: it commonly contains missing values, duplicate records, inconsistent formatting (like mixed date formats or inconsistent capitalization), and outliers caused by measurement or data-entry errors.

Skipping data cleaning risks building an analysis or model on flawed inputs — the well-known idea of 'garbage in, garbage out'. Careful cleaning (handling missing values deliberately, standardizing formats, removing or investigating duplicates) makes every later step more trustworthy.

A typical EDA workflow

A common workflow starts with basic inspection: checking the shape of the data, column data types, and the first few rows, followed by summary statistics (counts, means, min/max, missing-value counts) for each column.

Next comes univariate analysis — looking at one variable at a time with histograms, box plots, or bar charts — followed by bivariate or multivariate analysis, such as scatter plots or correlation heatmaps, to see how variables relate to each other and to any target variable of interest.

Throughout, the analyst asks questions of the data: Are there missing values, and are they random or systematic? Are there outliers, and are they errors or genuine extreme cases? Do relationships between variables match domain expectations?

The role of visualization

Visualization is central to EDA because summary statistics alone can hide important structure — famously, very different datasets can share nearly identical means, variances, and correlations while looking completely different when plotted (a phenomenon popularized by 'Anscombe's quartet').

Libraries like Matplotlib and Seaborn are commonly used for this stage in Python: Matplotlib provides fine-grained control over plots, while Seaborn builds on top of Matplotlib to make common statistical visualizations (distribution plots, box plots, pair plots, heatmaps) faster to produce with good default styling.